

Month 3: Purple Team Batch 01

Web App, API, & Cloud Security

Week 10: API Authentication Attacks & Advanced Testing

Instructor: **Muhammad Saad**

JWT token (JSON Web Token)

A **JWT token (JSON Web Token)** is a small piece of data used to **prove who you are** when you log into a website or app.

In simple words: It's like a **digital ID card** that your browser carries around after you log in.

How it works (very simply):

1. You **log in** (with username/password).
2. The server gives you a **JWT token**.
3. You send that token with every request (like opening pages or using features).
4. The server checks it and says:
"Yes, this user is already logged in."

What's inside it?

A JWT usually contains:

- Who you are (user ID or email)
- Some extra info (like role: admin/user)
- A digital signature (to prevent cheating)

Why it is used?

- You don't need to log in again and again
- It's fast and secure
- Common in modern apps (web + mobile)

Creation of JWT token (JSON Web Token)

A **JWT (JSON Web Token)** is made of **3 parts**, and they are separated by dots (.):

xxxxx.yyyyy.zzzzz

Header (First part)

Example: xxxxx

This part tells:

- What type of token it is (JWT)
- What algorithm is used to secure it (like HMAC SHA256 or RSA)

Simple idea:

It's like the **cover page of an ID card** saying how it is protected.

Payload (Second part)

Example: yyyyy

This contains the **actual data (claims)**, like:

- User ID
- Username or email
- User role (admin/user)
- Expiry time (when token becomes invalid)

Simple idea:

This is the **main information written inside the ID card**.

Signature (Third part)

Example: zzzzz

This is the **security part**.

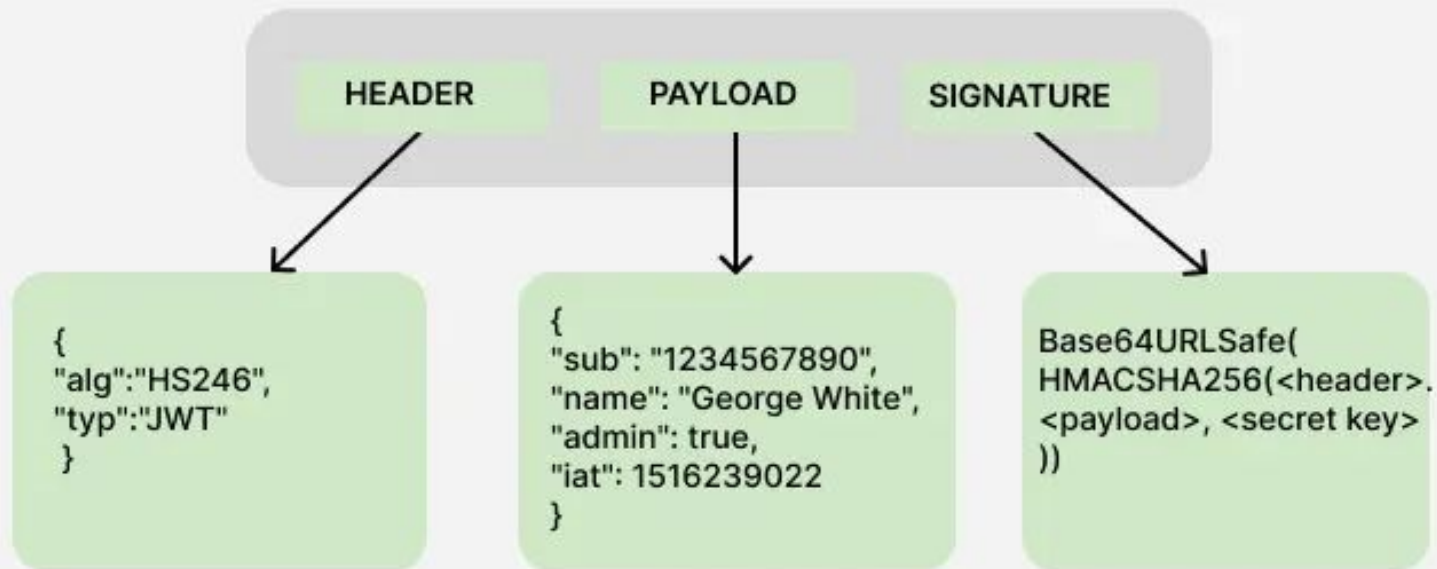
It is created using:

- Header + Payload
- A secret key (known only to server)

Purpose:

- To verify the token is **not changed or fake**
- If someone edits the payload, the signature will not match

Structure of a JSON Web Token (JWT)



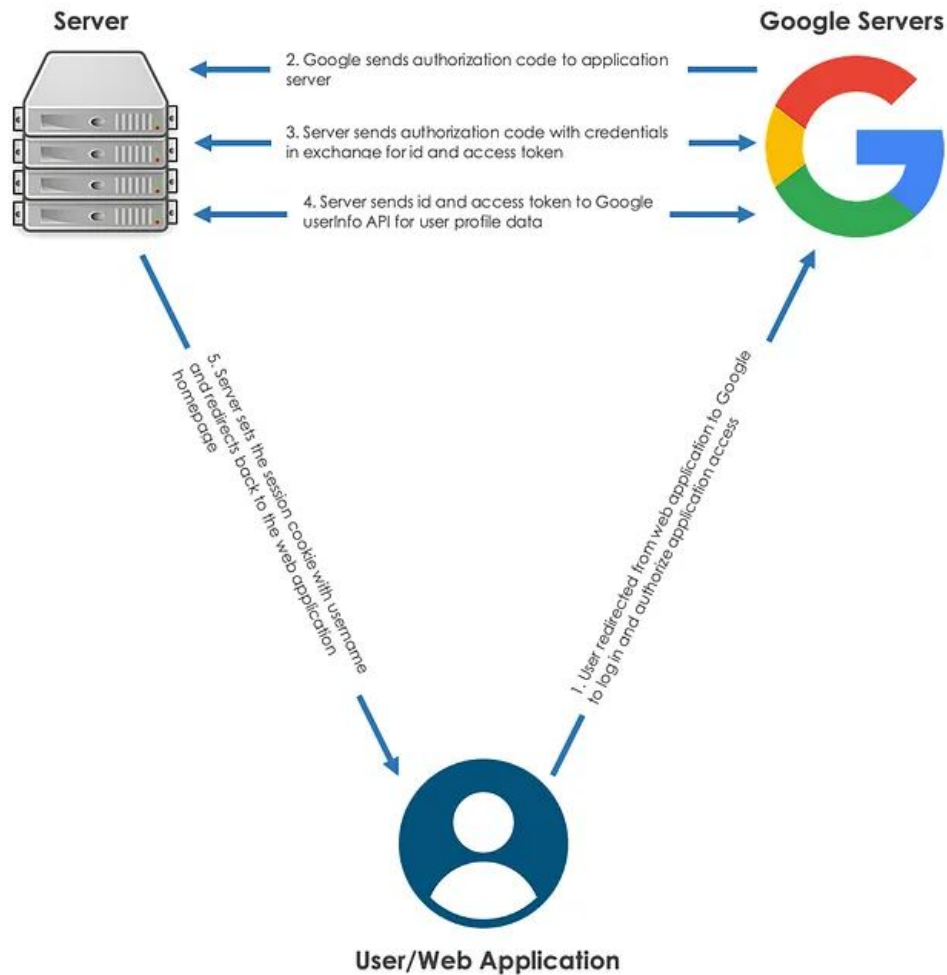
OAuth (Open Authorization)

OAuth (Open Authorization) is a way to let one app access your data from another app **without sharing your password**. It's widely used on the internet—like when you click “**Login with Google**” or “**Continue with Facebook**.”

Instead of giving your password to a third-party app, OAuth lets you say: “Okay, this app can access *some* of my data, but not everything.”

How OAuth Works (Step-by-Step)

1. You click “**Login with Google**.”
2. The app redirects you to Google.
3. You log in to Google (if not already).
4. Google asks: “Do you allow this app to access your email/profile?”
5. You click **Allow**.
6. Google gives the app a **token** (not your password).
7. The app uses that token to access your data.



API fuzzing

API fuzzing is a testing technique where you send **random, unexpected, or malformed data** to an API to see how it behaves.

Think of an API like a vending machine:

- Normally, you insert correct coins and press a button → you get a snack
- With fuzzing, you try weird things:
 - Insert broken coins
 - Press all buttons at once
 - Shake the machine

You're testing: *"Will it break? Misbehave? Give free snacks?"*

A tool or tester sends:

- Invalid inputs (wrong data types)
- Very large values
- Missing fields
- Extra/unexpected fields

API fuzzing Example

Suppose an API expects:

```
</> JSON  
  
{  
  "age": 25  
}
```

Fuzzing might send:

```
</> JSON  
  
{ "age": -999 }  
{ "age": "twenty-five" }  
{ "age": 9999999999999999 }  
{ "age": null }
```


Best of Luck for the Week 10 Tasks

Instructor: **Muhammad Saad**